

W15D02 — Request ID Tracing & Correlation

Phase 4 · Database & Observability | 16 Juni 2026

1. Konteks — Mengapa Request ID Dibutuhkan

Setelah W15D01 memasang infrastruktur logging (Logger → Handler → Formatter → Output), masalah baru muncul di production: saat ratusan request masuk secara bersamaan, semua log bercampur di satu stream tanpa penanda siapa yang memiliki baris log mana. Ketika ada error, mustahil men-trace perjalanan satu request dari awal hingga akhir.

Analogi: Rumah Sakit Tanpa Nomor Rekam Medis

Bayangkan sebuah rumah sakit besar di mana dokter, perawat, dan apoteker mencatat semua tindakan di satu buku besar bersama — tanpa nomor rekam medis. Catatan '10:02 pasien dirujuk ke IGD' tidak memberi tahu siapa pasiennya. Solusinya sederhana: setiap pasien yang masuk mendapat nomor rekam medis unik, dan semua catatan wajib mencantumkan nomor itu. Request ID adalah nomor rekam medis untuk setiap HTTP request.

Kondisi	Tanpa Request ID	Dengan Request ID
Log concurrent	Bercampur tak bisa dibedakan	Tiap baris berlabel UUID request
Debug error	Cari manual di ribuan baris	grep UUID → lihat journey lengkap
Distributed trace	Tidak mungkin	UUID sama bisa diteruskan ke service lain
Client korelasi	Tidak bisa tahu log mana miliknya	X-Request-ID di response header

2. ContextVar — Memori Terisolasi Per Request

ContextVar (dari modul contextvars, Python 3.7+) menyimpan nilai per execution context. Setiap async coroutine mendapat 'slot' terisolasi — bukan shared state. Ini kritis di FastAPI yang async: banyak request berjalan di satu event loop (single thread), sehingga threading.local() tidak cukup.

Analogi: Loker vs Papan Tulis Bersama

Variabel global seperti papan tulis bersama — satu pelayan menulis pesanan, pelayan lain langsung menghapusnya dan menulis pesanan pelanggan baru. Data milik pelanggan pertama hilang. ContextVar seperti loker pribadi — tiap pelayan punya laci sendiri, tidak bisa diakses oleh yang lain.

Cara Simpan State	Aman Concurrent?	Alasan
Global variable	Tidak	Semua request berbagi, saling overwrite
threading.local()	Partial	Aman per thread, tapi async pakai 1 thread
contextvars.ContextVar	Ya (pilihan tepat)	Terisolasi per coroutine/task, async-safe

3. Arsitektur Implementasi

Tiga komponen bekerja bersama — masing-masing punya satu tanggung jawab (Separation of Concerns, W11):

- context.py — Definisi ContextVar. Di-import middleware (set) dan filter (get). Tidak ada logic lain.
- RequestIDMiddleware — Membuat UUID, menyimpan ke ContextVar, mengukur durasi, menambah response header.
- RequestIDFilter — Dipanggil setiap kali ada LogRecord. Inject request_id dari ContextVar ke record.

Alur per request: Client → Middleware (buat UUID, set ContextVar) → Router/Service (logger.info/error seperti biasa) → Logger memanggil Filter (inject request_id) → Handler menulis ke stdout → Response kembali ke client dengan header X-Request-ID.

4. Implementasi Kode

app/core/context.py

```
from contextvars import ContextVar

# Slot terisolasi per coroutine. Default "system" untuk log
# yang terjadi di luar siklus request (startup, shutdown).
request_id_context_var: ContextVar[str] = ContextVar("request_id", default="system")
```

app/middleware/request_id.py

```
import uuid, time, logging

from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import Response
from app.core.context import request_id_context_var

logger = logging.getLogger(__name__)

class RequestIDMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next) -> Response:
```

```

# Pakai ID dari header jika ada (distributed tracing), buat baru jika tidak
request_id = request.headers.get("X-Request-ID") or str(uuid.uuid4())

# set() mengembalikan token – diperlukan untuk reset() nanti
token = request_id_context_var.set(request_id)

start_time = time.perf_counter()

try:
    response = await call_next(request)
    duration_ms = round((time.perf_counter() - start_time) * 1000, 2)
    logger.info(f"{request.method} {request.url.path} completed | duration_ms={duration_ms}")
except Exception:
    logger.error("Unhandled exception in request", exc_info=True)
    raise
finally:
    # WAJIB di finally – berjalan meski ada exception
    # Tanpa ini: ContextVar 'kotor' bocor ke request berikutnya
    request_id_context_var.reset(token)

response.headers["X-Request-ID"] = request_id

return response

```

app/core/logging_config.py (update dari W15D01)

```

import logging, sys

from app.core.context import request_id_context_var

class RequestIDFilter(logging.Filter):
    def filter(self, record: logging.LogRecord) -> bool:
        # Inject request_id ke setiap LogRecord sebelum sampai ke Handler
        record.request_id = request_id_context_var.get()
        return True # True = record diteruskan, False = di-drop

def setup_global_logging() -> None:
    root_logger = logging.getLogger()

    if root_logger.hasHandlers(): # Guard: cegah duplicate handler
        return

```

```

root_logger.setLevel(logging.DEBUG)

handler = logging.StreamHandler(sys.stdout)
handler.setLevel(logging.INFO)
# %(request_id)s membaca atribut yang di-set RequestIDFilter
handler.setFormatter(logging.Formatter(
    fmt="%(asctime)s | %(levelname)-8s | %(request_id)s | %(name)s | %(message)s",
    datefmt="%Y-%m-%dT%H:%M:%S"
))
handler.addFilter(RequestIDFilter()) # Filter dipasang ke Handler, bukan Logger
root_logger.addHandler(handler)

logging.getLogger("uvicorn.access").setLevel(logging.WARNING)
logging.getLogger("sqlalchemy.engine").setLevel(logging.WARNING)

```

main.py — Urutan Kritis di Lifespan

```

from contextlib import asynccontextmanager
from fastapi import FastAPI
from app.core.logging_config import setup_global_logging
from app.middleware.request_id import RequestIDMiddleware
import logging

logger = logging.getLogger(__name__) # Deklarasi di module level – aman, belum memanggil handler

@asynccontextmanager
async def lifespan(app: FastAPI):
    setup_global_logging() # PERTAMA – sebelum logger.info apapun
    logger.info("Memulai API...") # Baru setelah setup selesai
    yield
    logger.info("Mematikan server...")
    await async_engine.dispose()

app = FastAPI(lifespan=lifespan)
app.add_middleware(RequestIDMiddleware) # LIFO: didaftarkan terakhir, berjalan pertama

```

5. Hal-Hal Kritis yang Harus Diingat

A. `setup_global_logging()` harus di dalam lifespan

Jika dipanggil di luar lifespan (di module level), fungsi ini berjalan saat Python mengimport file — sebelum environment variables terbaca, sebelum FastAPI fully initialized. Ini menyebabkan konfigurasi yang tidak dapat diandalkan.

B. token dari `set()` wajib disimpan dan dipakai di `reset()`

`request_id_context_var.set()` mengembalikan Token yang merepresentasikan state `ContextVar` sebelum di-set. `reset(token)` mengembalikan `ContextVar` ke state tersebut. Tanpa token, `reset()` tidak bisa dipanggil — state bocor ke coroutine berikutnya.

C. `reset()` wajib di blok `finally`

Jika exception terjadi di `await call_next(request)`, eksekusi melompat ke `except` dan tidak pernah mencapai baris setelahnya. Blok `finally` selalu dieksekusi — baik dalam kondisi normal maupun ketika ada exception. Ini memastikan tidak ada state yang 'menggantung' setelah request selesai.

D. Background Tasks mewarisi Context secara otomatis

FastAPI BackgroundTasks mengambil snapshot context saat task dijadwalkan — termasuk nilai `ContextVar` saat itu. Ini berarti background task berjalan dengan `request_id` yang sama dengan request yang memicunya, meski middleware sudah reset. Ini bukan bug — ini feature Python contextvars yang dirancang untuk tracing.

E. Middleware FastAPI menggunakan pola LIFO

`app.add_middleware()` yang dipanggil TERAKHIR akan BERJALAN PERTAMA saat request masuk. Jika ada `AuthMiddleware` dan `RequestIDMiddleware`, daftarkan `RequestIDMiddleware` setelah `AuthMiddleware` agar `request_id` sudah tersedia ketika auth diproses.

F. X-Request-ID header untuk client correlation

Menambahkan X-Request-ID ke response header memungkinkan client (Postman, frontend, service lain) mengkorelasikan response mereka dengan log server. Standar industri ini juga mendukung distributed tracing: service yang menerima request dapat meneruskan header yang sama ke downstream service, sehingga satu request dapat di-trace melewati beberapa service sekaligus.

6. Contoh Output Log

```
# Log startup (sebelum request masuk) – request_id = default "system"
```

```
2026-06-16T13:15:02 | INFO | system | main | Memulai Data Science System API...
```

```
# Log saat request masuk – request_id = UUID yang dibuat middleware
```

```
2026-06-16T13:15:20 | INFO | 4ae0ddaf-8b7e-4b75-bec8-fa5b4e9c5009 | app.routers.predictions  
| Menerima request
```

```
2026-06-16T13:15:20 | INFO | 4ae0ddaf-8b7e-4b75-bec8-fa5b4e9c5009 |  
app.middleware.request_id | POST /predictions/ completed | duration_ms=150.91
```

```
# Background task — mewarisi UUID dari request yang memicunya (expected behavior)
```

```
2026-06-16T13:15:35 | INFO | 4ae0ddaf-8b7e-4b75-bec8-fa5b4e9c5009 |  
app.services.analysis_services | Selesai komputasi ML
```

Untuk men-trace satu request: `grep '4ae0ddaf'` pada log → lihat seluruh perjalanan request tersebut, termasuk background task yang dipicunya.

7. Ringkasan Knowledge Check

Q1 — Log startup dengan '-' atau 'system'

Log pertama saat server hidup memiliki `request_id` default karena tidak ada request HTTP yang masuk — middleware belum pernah dipanggil. `ContextVar` mengambil nilai default yang sudah ditetapkan ('system' atau '-'). Ini perilaku yang benar dan diharapkan.

Q2 — Kenapa global variable berbahaya di async

Di lingkungan concurrent, global variable adalah papan tulis bersama: request A menulis nilainya, request B langsung menimpa. Ketika response A siap, ID yang tertulis sudah bukan milik A lagi. `ContextVar` memberikan 'loker pribadi' per coroutine.

Q3 — Bahaya tidak menyimpan token

Tanpa token dari `set()`, `reset()` tidak bisa dipanggil. Tanpa `finally`, exception menyebabkan `reset()` tidak pernah tercapai. Hasilnya: `ContextVar` ditinggalkan dalam keadaan kotor — request berikutnya yang kebetulan mendapat coroutine yang sama akan membaca `request_id` milik request sebelumnya (kontaminasi silang). Ditambah: tanpa `finally`, setiap exception menyebabkan memory leak bertahap.